

Project Report
Systems Validation - 2IMF30

Automated storage system

A. Novokhatskiy (2161699) C. Milanese (2039648) M. Stoian (1539779)

September 28, 2026

1 System Description

The system presented in this report and modelled with mCRL2 is an automated storage facility.

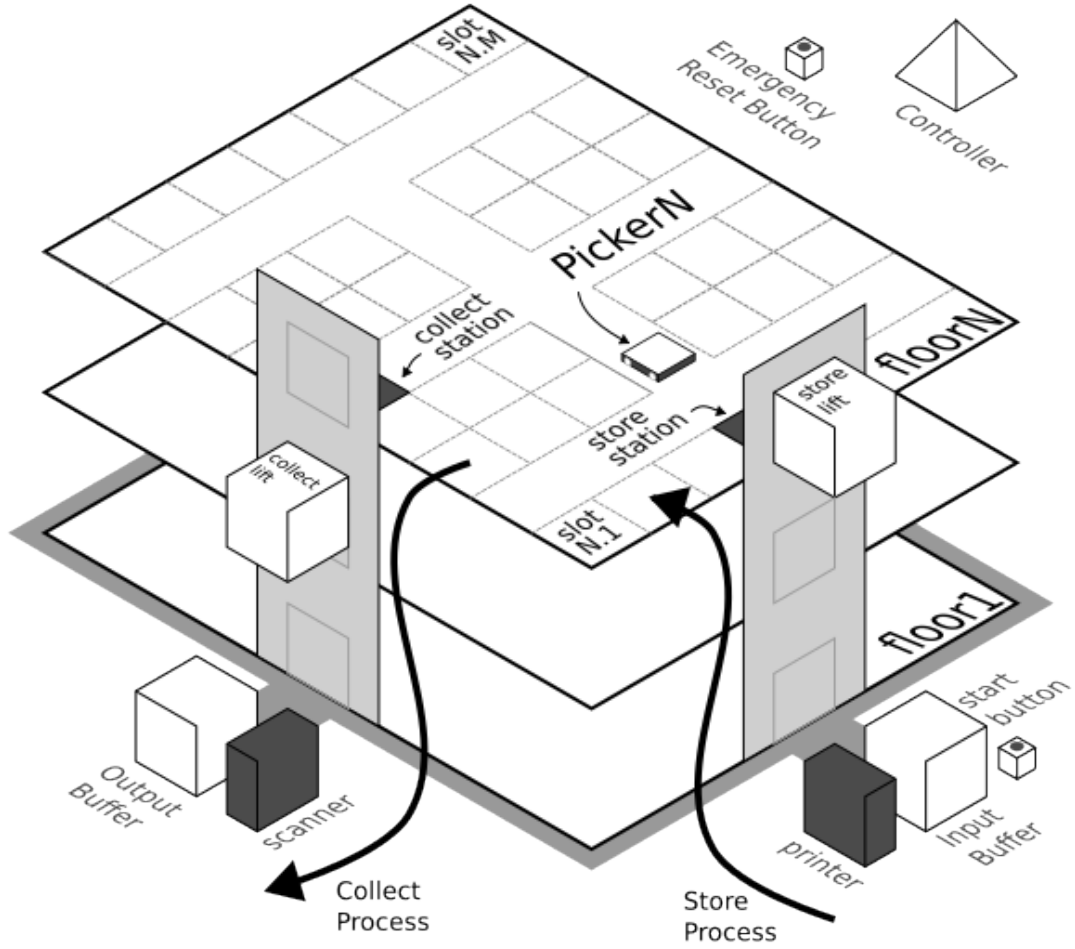


Figure 1: Illustration of the system composition.

2 Requirements in plain English

To ensure that the system described in this report functions as intended, its behavior is subject to requirements. This section contains the definition of those requirements expressed in plain English.

1. When the start button is pressed with an item in the input buffer and storage is not full, the printer eventually provides a ticket.
2. The printer never provides a ticket when storage is full.
3. A ticket is never issued for an item before a label has been applied to that item.
4. Once a ticket is provided by the printer, the item will eventually be placed at the assigned slot, unless an emergency occurs.
5. When the system is in the emergency state, neither the store lift, the collect lift, nor any picker moves.
6. After the emergency reset button is pressed, every store or collect operation interrupted by the emergency eventually completes.
7. A picker never places an item in an occupied slot.
8. A picker never attempts to pick up an item from an empty slot.
9. A picker never attempts to place an item in a slot that it is not adjacent to.
10. An item is loaded onto the store lift only if the store lift is at the ground floor.
11. The store lift can hand over an item to a picker only if the store lift is at that picker's floor and the picker is at the store station.
12. The store lift never moves while a handover with a picker is in progress.
13. After handing over an item to a picker, the store lift eventually returns to the ground floor, unless an emergency occurs.
14. After receiving an item, the picker eventually places it in the slot assigned to that item, unless an emergency occurs.
15. A ticket is never issued for slot that is currently occupied or a picker is in the process of depositing an item into that slot.
16. When the scanner reads a valid ticket, the item at the slot assigned on that ticket is eventually delivered to the output buffer, unless an emergency occurs.
17. A picker can hand over an item to the collect lift only if the collect lift is at that picker's floor and the picker is at the collect station.

3 Actions

Action	Description
Lift	
<i>moveUp()</i>	The lift moves to the floor above.
<i>moveDown()</i>	The lift moves to the floor below.
<i>moveTo(N)</i>	The lift goes to floor N.
<i>pickupItem()</i>	The lift picks up the item at its current floor.
<i>dropOffItem()</i>	The lift places down the picked up item at its current floor.
Emergency	
<i>triggerEmergency()</i>	An emergency is triggered, all actions are prevented until it is reset.
<i>resetEmergency()</i>	The emergency is reset, processes resume actions.
Input Buffer	
<i>itemUserIn()</i>	The user places an item into the input buffer.
<i>itemLiftOut()</i>	The item inside the input buffer is transferred to the lift.
Output Buffer	
<i>itemLiftIn()</i>	The lift places an item into the output buffer.
<i>itemUserOut()</i>	The item inside the output buffer is transferred to the user.
Picker	
<i>collectFromLift()</i>	The picker picks up the item from the lift.
<i>giveToLift()</i>	The item on the picker is transferred to the lift.
<i>goToSlot(N, M)</i>	The picker physically moves to the storage slot M on its current floor N.
<i>storeItemToSlot(N, M)</i>	The picker places its item in the slot M on floor N.
<i>collectItemFromSlot(N, M)</i>	The picker picks up the item in the slot M on floor N.
<i>storeItemToSlot() + TBD Q4</i>	The picker places its item at the current position.
<i>collectItemFromSlot()</i>	The picker picks up the item at the current position.
<i>goToCollectStation()</i>	The picker physically moves to the collection station on its floor.
<i>goToStoreStation()</i>	The picker physically moves to the store station on its floor.
Start Button	

<i>buttonPressed()</i>	The start button is pressed.
Printer	
<i>printTicket(N, M)</i>	The ticket N,M is printed and issued to the user.
<i>putLabel(N, M)</i>	A label is placed on the item marking it as stored on floor N slot M.
<i>showStorageFull()</i>	An error is shown to the user (for example via a screen) that the storage is full.
Scanner	
<i>scanTicket(N,M)</i>	The ticket N,M presented by the user is scanned.
<i>showTicketInvalid()</i>	An error is shown to the user that the item is not present.

4 Requirements with actions.

1. When *buttonPressed()* after *itemUserIn()* and not *showStorageFull()*, the printer eventually does *printTicket(N, M)*.
2. The printer never does *printTicket(N, M)* when storage is full.
3. The printer does *printTicket(N, M)* only after *putLabel(N, M)*.
4. Once *printTicket(N, M)* is done by the printer, the picker will eventually *storeItemToSlot(N, M)* at the assigned slot, unless *triggerEmergency()* occurs.
5. When *triggerEmergency()* happens, no action happens before *resetEmergency()*.
6. After *resetEmergency()*, every store or collect operation interrupted by the emergency eventually completes.
7. A picker never places an item in an occupied slot. **Seems redundant after 4**
8. A picker will *collectItemFromSlot(N,M)* **once**, only after *storeItemToSlot(N,M)* has occurred for the same N and M.
9. A picker will *collectItemFromSlot(N,M)* or *storeItemToSlot(N,M)* only after *goToSlot(N,M)* has occurred for the same N and M.
10. The *itemLiftIn()* happens only if collection lift cannot *moveDown()*.
11. The *itemLiftOut()* happens only if the store lift cannot *moveDown()*.
12. The store lift can hand over an item to a picker only if the store lift is at that picker's floor and the picker is at the store station. **I believe having action "goToFloor(N)" instead of "moveUp()" and "moveDown()" would allow us to simplify this requirement to: The store lift does *dropOffItem(N)* only after the picker has done *goToStoreStation(N)* and the store lift *goToFloor(N)* for the same N.**
13. The store lift never moves while a handover with a picker is in progress. **Maybe something like "goToFloor(N) can't happen twice without a *dropOffItem()* or *pickupItem()* in between" which would be valid for any lift**
14. After *dropOffItem()*, the store lift eventually *goToFloor(0)*, unless *triggerEmergency()* occurs.
15. After receiving an item, the picker eventually places it in the slot assigned to that item, unless an emergency occurs.
16. The printer will not do *printTicket(N,M)* twice for the same N and M, without a *scanTicket(N,M)* in between.
17. When *scanTicket(N,M)* occurs, the output buffer will eventually *itemLiftIn()*, unless *triggerEmergency()* occurs.
18. A picker at floor N will *giveToLift()* only after *goToCollectStation()* and the collect lift did *goToLift(N)* for that N.

5 Component definitions in mCRL2

```
map numFloors : Nat;  
eqn numFloors = 3;  
  
sort Item;  
cons hasItem , noItem : Item;
```

5.1 Lift component

```
act pickUpItem , dropOffItem : Nat;  
    moveToFloor : Nat # Nat;  
  
proc Lift(F: Nat, I: Item, N: Nat) =  
    pickUpItem(N).Lift(F, hasItem, N) +  
    dropOffItem(N).Lift(F, noItem, N) +  
    sum f: Nat . (f > 0 && f < numFloors && f != F)  
        -> moveToFloor(f, N).Lift(f, I, N);
```

5.2 Emergency and Start button component

```
act triggerEmergency , resetEmergency , startPress;  
  
proc EmergencyButton = triggerEmergency . resetEmergency . Emergency;  
    StartButton = startPress . StartButton;
```

5.3 Input and output buffers

```
act itemInUser , itemInLift , itemOutLift , itemOutUser;  
  
proc  
    InputBuffer(I: Item) = (I == hasItem)  
        -> itemInUser . InputBuffer(hasItem)  
        <> itemOutLift . InputBuffer(noItem);  
  
    OutputBuffer(I: Item) = (I == hasItem)  
        -> itemInLift . OutputBuffer(hasItem)  
        <> itemOutUser . OutputBuffer(noItem);
```

6 Questions and answers for TA meeting

- If a component type has multiple instances, will all its actions get a parameter to identify the instance doing that action?
- Do we need to model the allocations of packets in the system by indexing?
- Lift is a single component, doing actions *pickupItem()* and *dropOffItem()*. Yet one lift only picks up from the buffer and the other only from the picker (station), do we need to disambiguate?
- How to properly parametrize the picker on each floor. Eg is it 3 components or calling components with first parameter being floor(*goToSpot(floor, M, N)* where M,N are the coordinates of the spot)

6.1 Meeting 24-09

- Do we need an extra component to decide whether the storage is full or not?
- once we have all the parallel components, do we need to disallow single actions and allow only the combined one? S: rename (comm operator) two actions that happens in parallel into a single one and allow only the combined one.

7 Next to do

.....